

* Why move to Spring Boot?

In standard Spring Core development, the developer takes on several repetitive, manual responsibilities:

- Manual Setup → Initializing the ApplicationContext (IOC) manually.
- Explicit Configuration → Defining configuration classes with @Configuration, @ComponentScan and @Bean methods to manage dependencies.
- Dependency Injection → Manually ensuring all beans are identified and wired correctly.

* Spring Boot Difference :-

- Spring Boot introduces Auto-Configuration, which significantly reduces the amount of boilerplate code.
- It is not a replacement for Spring Core; rather it is an intelligent startup system built on top of Spring Core.
- You still use standard annotations like @Component and @Autowired, but Spring Boot automates the surrounding infrastructure, allowing you to focus purely on business logic.
- You simply add @SpringBootApplication annotation to your main class, and it automatically scans the package for components and manages the IOC container for you.

* Setting up a Spring Boot Project :-

Moving from manual dependency management to Spring Boot involves two primary ways to structure your project :-

1. Spring-Boot-starter approach →

Instead of managing every individual dependency, you use a "starter" dependency. A starter acts as a curated bundle of dependencies (transitive dependencies), which simplifies your `pom.xml` significantly.

2. Spring-Boot-starter-parent →

This parent POM handles version management for you.

When you use this parent, you no longer need to specify version numbers for your dependencies. Spring Boot ensures that all versions are compatible with each other, preventing the "dependency hell" of version mismatches.

Spring Initializr → The standard tool to generate a template project with all ~~unnecessary~~ configurations already set up.

Consistency → The parent POM makes your project much easier to maintain as it grows, especially when adding modules like Spring Data or Spring Web.

* Understanding @SpringBootApplication :-

This core annotation is what makes Spring Boot magic. It is actually a convenient combination of three distinct annotations :-

1. @SpringBootConfiguration → This marks your main class as a source of bean definitions, functioning exactly like old @Configuration in Spring Core. This allows you to define @Bean methods directly in your main file if needed.
2. @ComponentScan → This instructs Spring to search the current package and its sub-packages for any classes annotated with @Component, @Service, etc. and automatically registers them as beans.
3. @EnableAutoConfiguration → This is the most powerful part of it. It tells Spring Boot to scan your classpath (dependencies in pom.xml) and automatically configure beans based on what it finds.

→ How it works together :-

- When you run the main method, Spring Boot

initializes the IOC container, performs the scan, and applies those configurations automatically.

- You no longer need to write a manual `ApplicationContext` or explicit XML config file; Spring Boot handles the heavy lifting based on these conventions.

* Deep-Dive into Auto-Configuration :-

The most powerful feature of Spring Boot is its ability to intelligently configure your application based on the dependencies present in your classpath. This is managed via '@EnableAutoConfiguration' annotation and specific logic to prevent conflicts.

How Auto-Configuration Works Internally →

- @AutoConfiguration → While you rarely use this directly, this annotation marks specific internal classes within Spring Boot that contain pre-defined bean configurations. These classes tell Spring which objects (like Tomcat or Database drivers) to automatically create and manage.

Date.....

- Conditional Logic → Spring Boot uses specific annotations to decide whether to activate these beans, ensuring your application only runs what it needs:

1. @ConditionalOnClass: A bean is created only if a specific library (a class) file exists in your project's dependencies. This is why Tomcat only starts when you add 'spring-boot-starter-web' dependency.

2. @ConditionalOnMissingBean: A bean is created only if the user hasn't already defined their own version of that bean. This gives you full control - if you define your own, Spring Boot steps aside.

* External Configurations :-

In modern enterprise applications, hardcoding values (like API keys, database URLs, or retry counts) inside Java classes is a bad practice. It forces you to recompile the code whenever a configuration changes. Spring Boot solves this with externalized configuration.

→ What is application.properties?

It is a non-Java configuration file located in 'src/main/resources' folder. It uses a simple [key = value] format, acting like a map for your application's settings.

→ Injecting values with @Value

If you want to inject a single property into a class, use the @Value annotation with the syntax ``${property.name}``.

Example ->

```
@Value("${payment.gateway.type}")  
private String type;
```

```
@Value("${payment.gateway.type:Razorpay}")  
private String type;
```

Date.....

- DefaultValue : You can provide a fallback value if the property is missing using `defaultValue`.

→ Problem with @Value

Using `@Value` for every single field becomes messy and error-prone as the project grows. This is where `@ConfigurationProperties` becomes necessary for managing grouped properties efficiently.

* @ConfigurationProperties :-

When a class contains many related settings, using individual `@Value` annotations can become cluttered and hard to maintain.

→ Setting up @ConfigurationProperties

Instead of injecting variables one-by-one, you create a dedicated class (e.g. `PaymentProperties`) and map it to a prefix defined in `application.properties`.

- Prefix Syntax → Use a canonical format (kebab-case) in your application properties file, like `payment-property.type`.

- class mapping → Annotate the class with `@ConfigurationProperties` (prefix = "payment-property").

Example 5- (application.properties)

```
payment-property.type = Paytm
payment-property.retry-count = 5
payment-property.enabled = true
payment-property.time-out = 3000
```

(Java Class)

@Component

@ConfigurationProperties("payment-property")

```
public class PaymentProperties {
    private String type;
    private int retryCount;
    private boolean enabled;
    private int timeout;
```

→ Spring Boot is highly flexible. Even if your Java variables use camelCase (e.g. retryCount), Spring Boot can automatically map them to kebab-case properties in config file (e.g. retry-count).

* Executing code at Startup :-

Sometimes you need to run specific code or logic automatically when a Spring Boot application starts, without ~~knowing~~ waiting for an external HTTP request (like a controller endpoint).

→ Need for Runner Interface :-

In standard Spring Boot development, we avoid manually calling beans from the main method. Instead, we use Runner interfaces, which allow the application to execute custom code automatically during the startup process.

→ CommandLineRunner Vs. ApplicationRunner :-

Both are functional interfaces used to execute ~~code~~ code immediately after the application context is loaded.

- CommandLineRunner → Used a run (String... args) method. It takes a variable length argument list (varargs), which is useful if you want to pass raw command-line arguments to your logic.

- ApplicationRunner → Uses a `run(ApplicationArguments args)` method. It provides a more structured 'ApplicationArguments' object, making it easier to handle command-line options and non-option arguments.

→ Example: To execute code, create a component class that implements one of these interfaces:-

@Component

```
public class DemoRunner implements ApplicationRunner {
```

@Autowired

```
private PaymentGateway paymentGateway;
```

@Override

```
public void run(ApplicationArguments args) throws  
Exception {
```

```
    paymentGateway.print();  
}
```

```
}
```

// Automatically runs at startup

Beyond the IDE, you can trigger these startup sequences via the command line using
[`mvn: mvn spring-boot:run`]